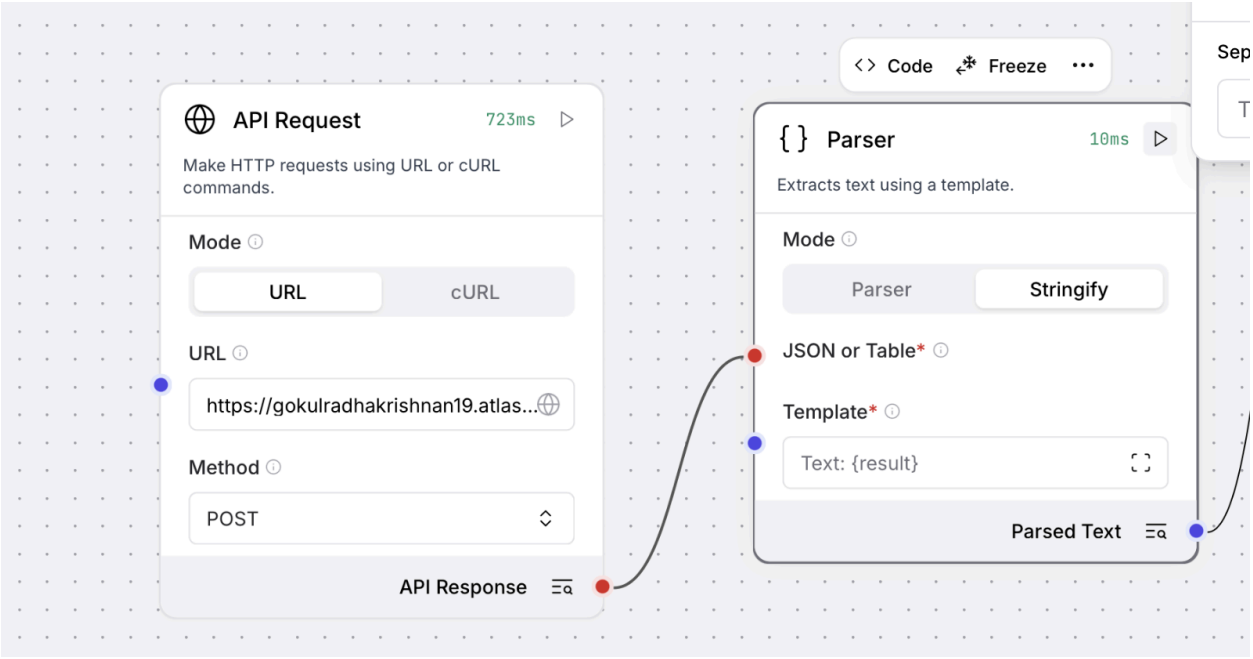
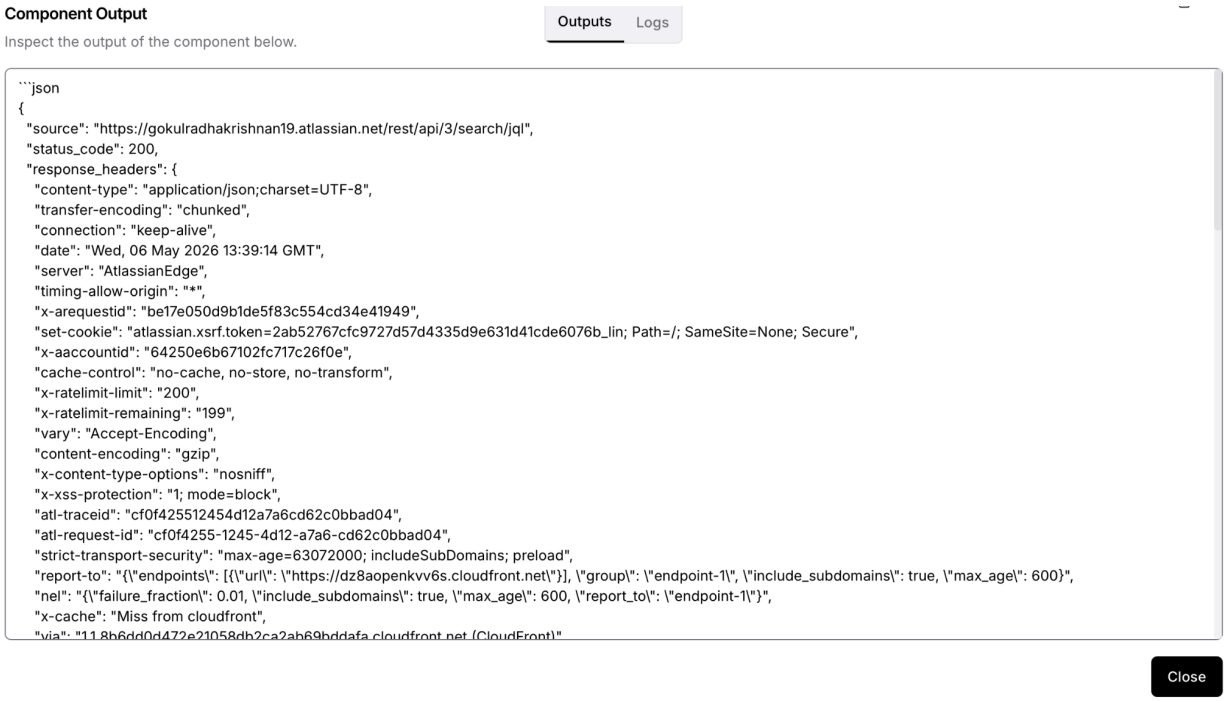
Week 6 Day-1 Assignment-1

A) User Story Review Agent With Langflow:

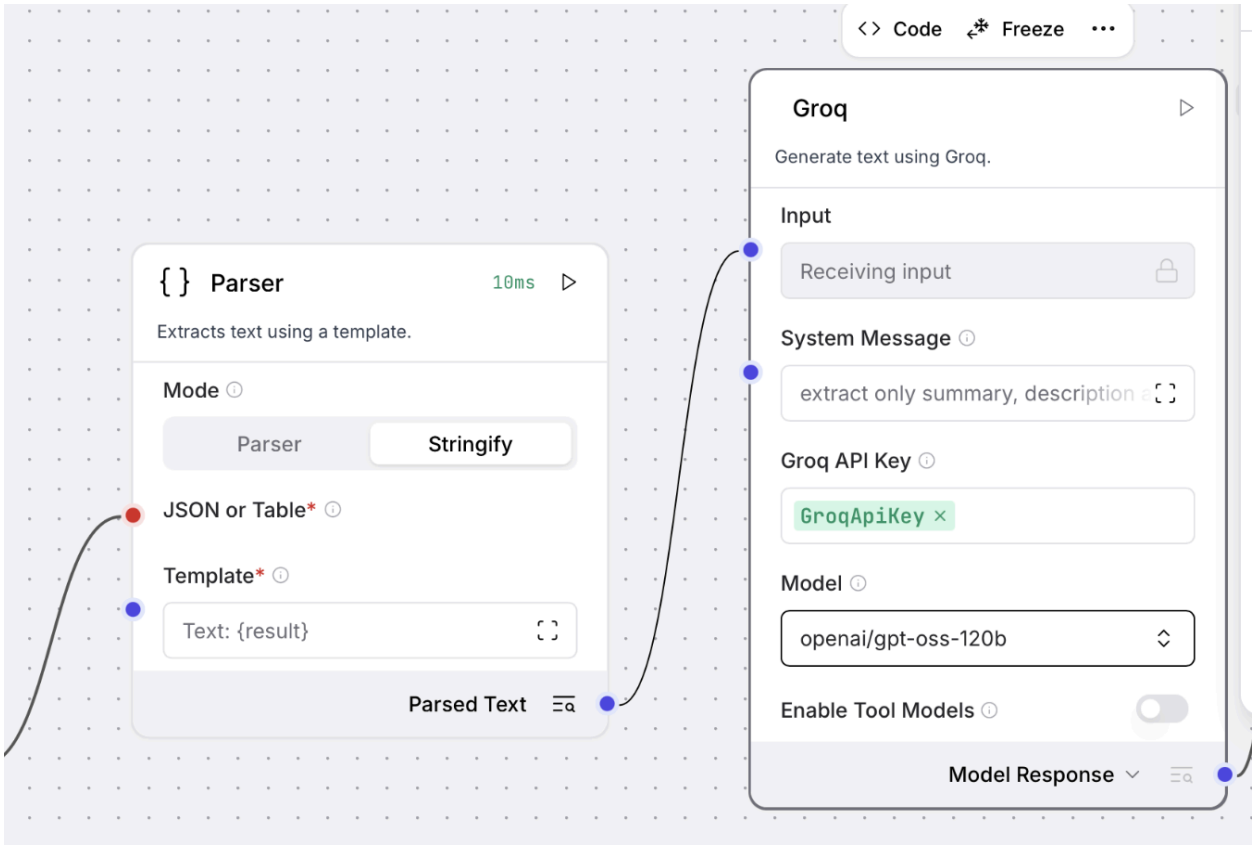
Api request connect to Parser:



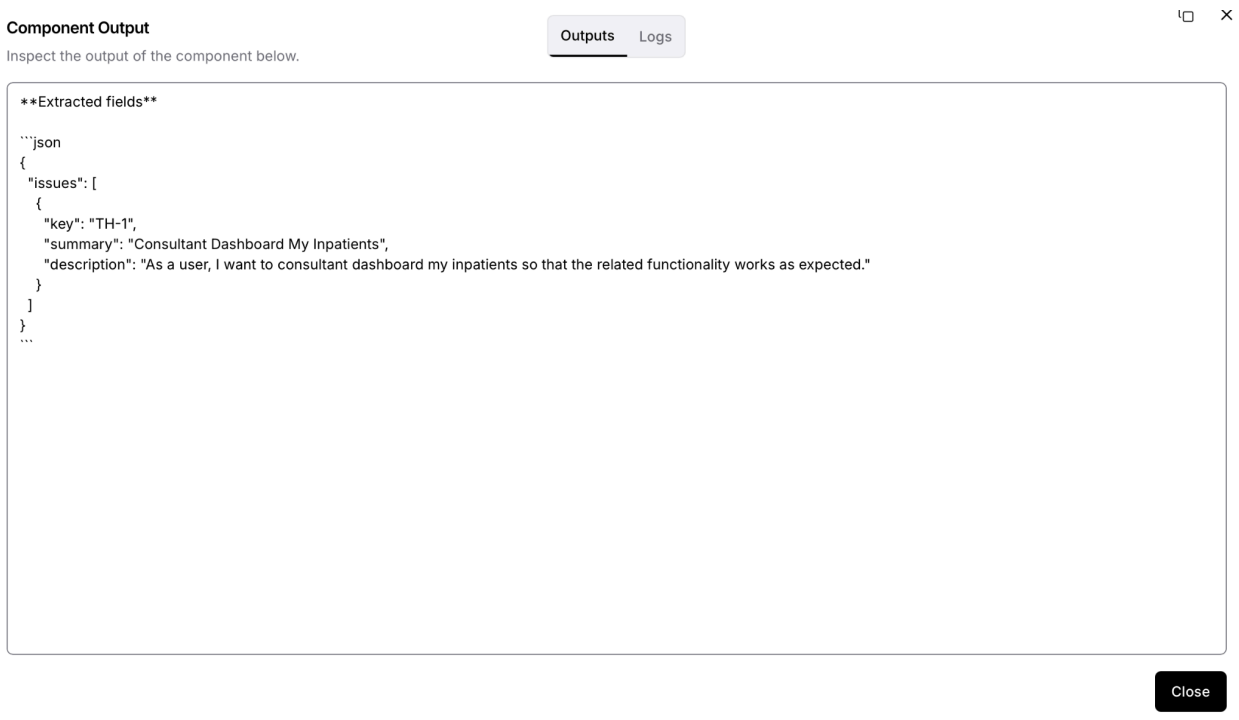
Parser Output:



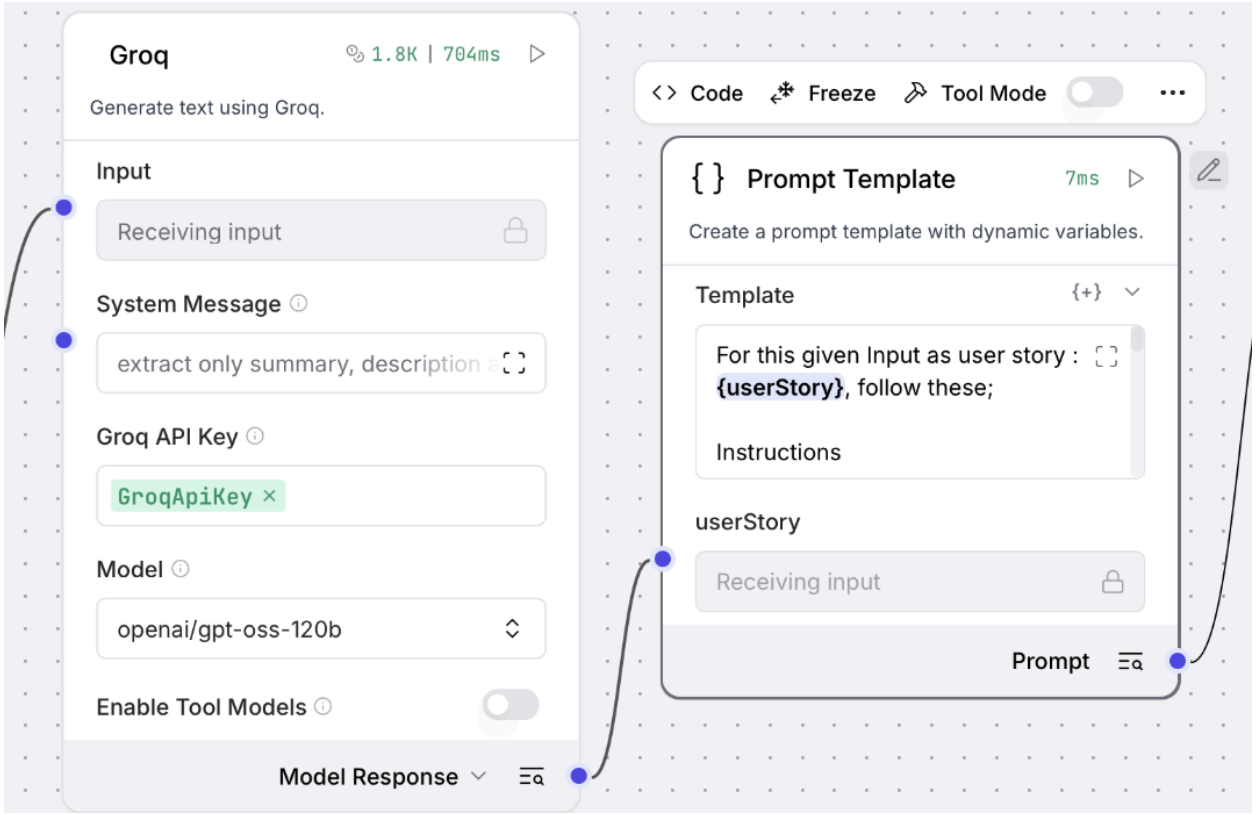
Parser to Groq:



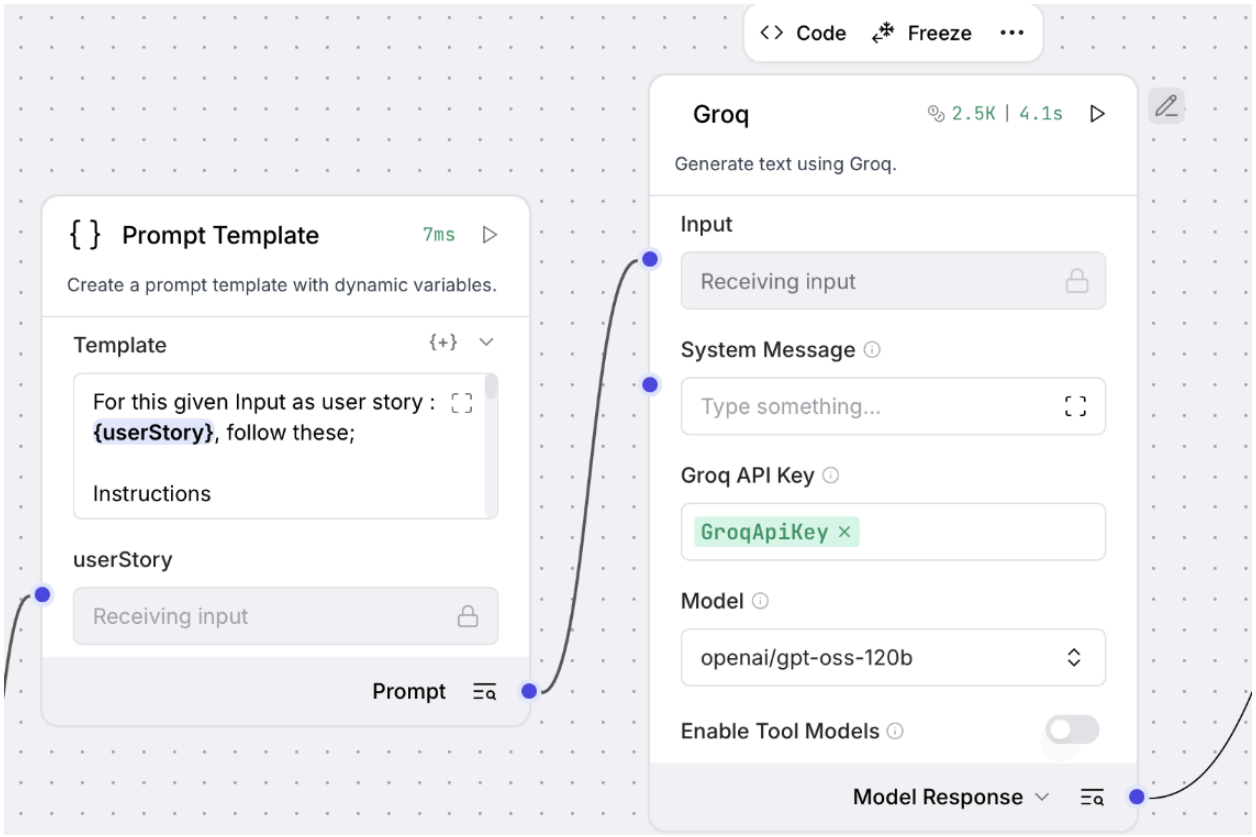
Groq Output:



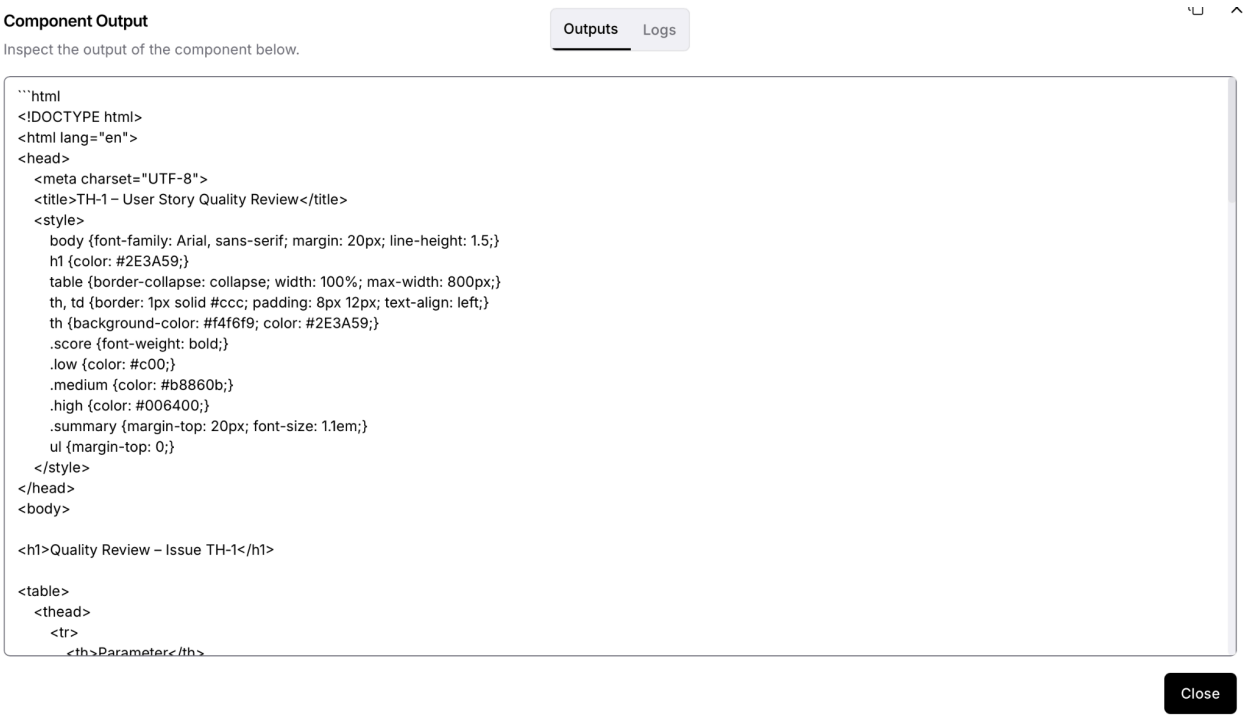
Groq to prompt template:



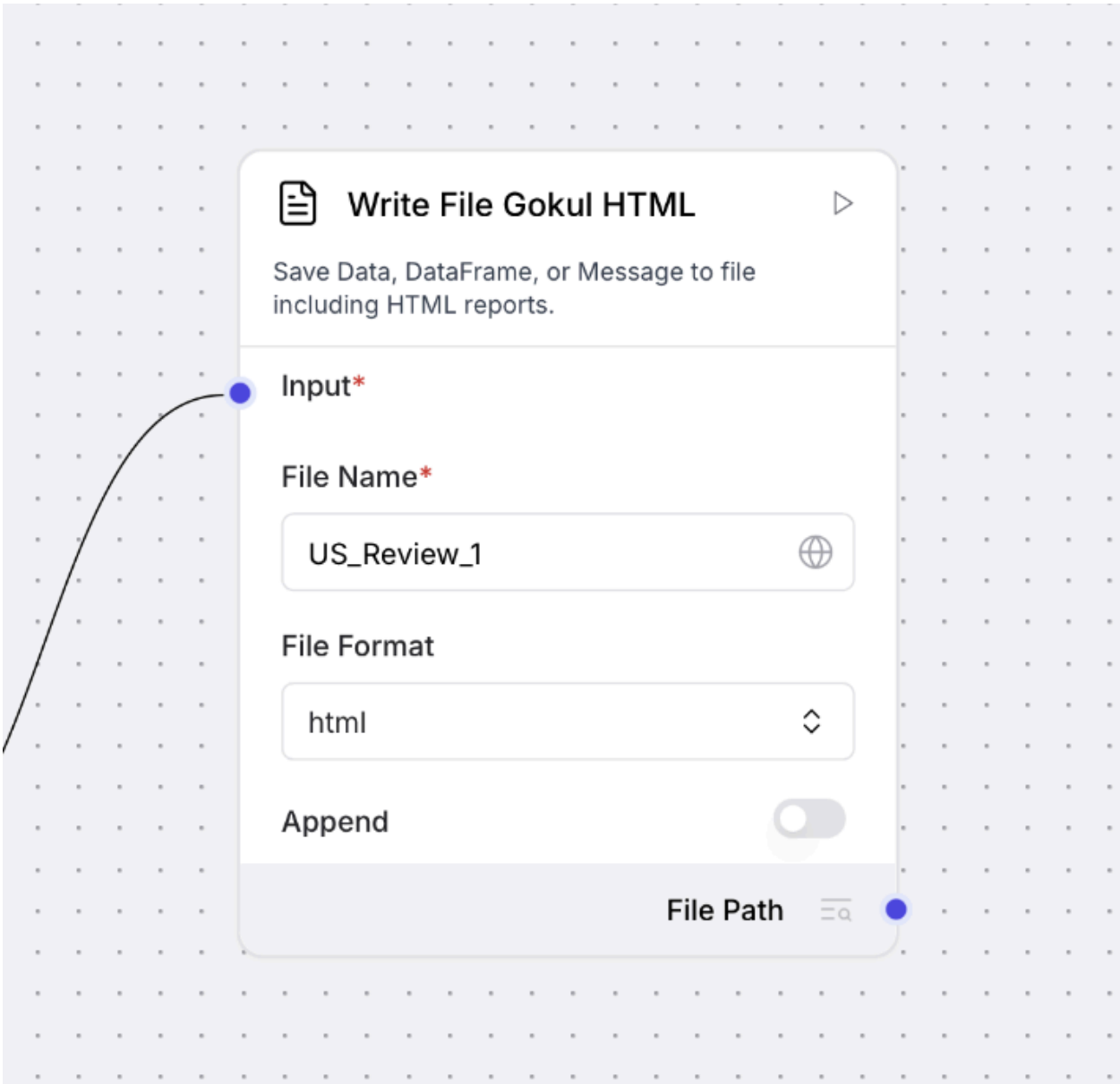
Prompt to Groq again:

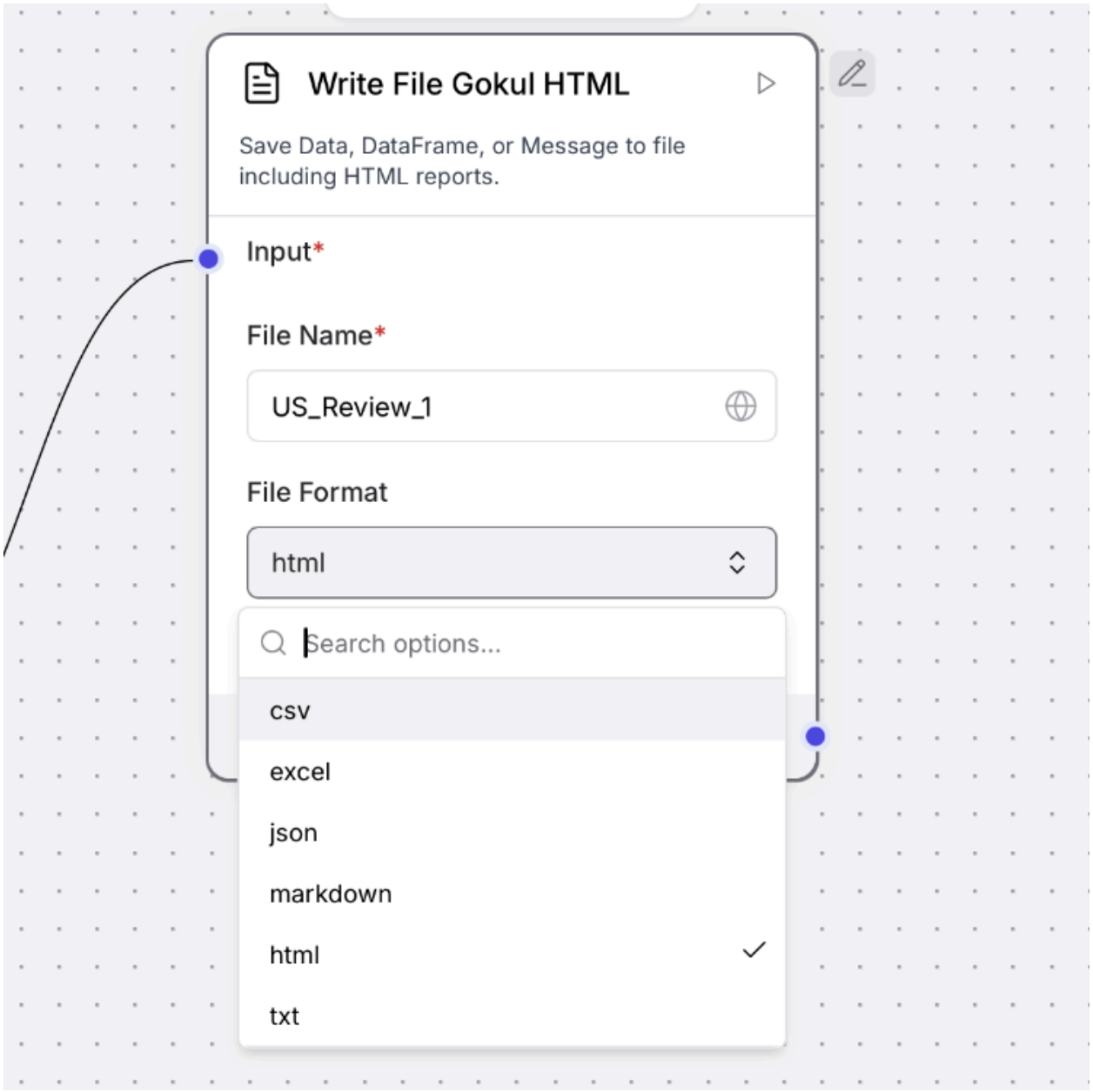


Html output from Groq:

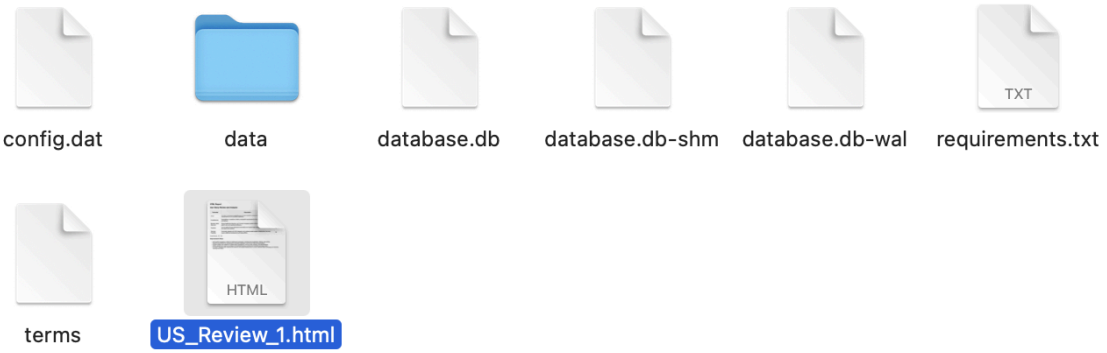


Custom component created by modifying the python code to inclue html option:

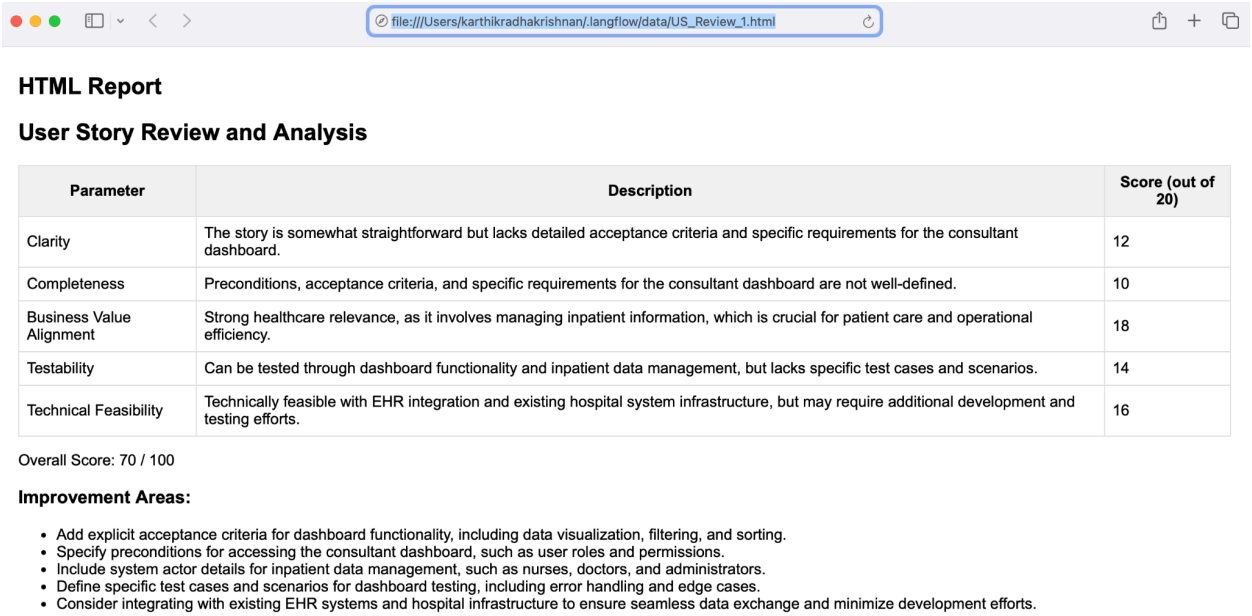



Write File Gokul HTML
Save Data, DataFrame, or Message to file including HTML reports.
Input*
File Name*
US_Review_1
File Format
html
Search options...
csv
excel
json
markdown
html
txt

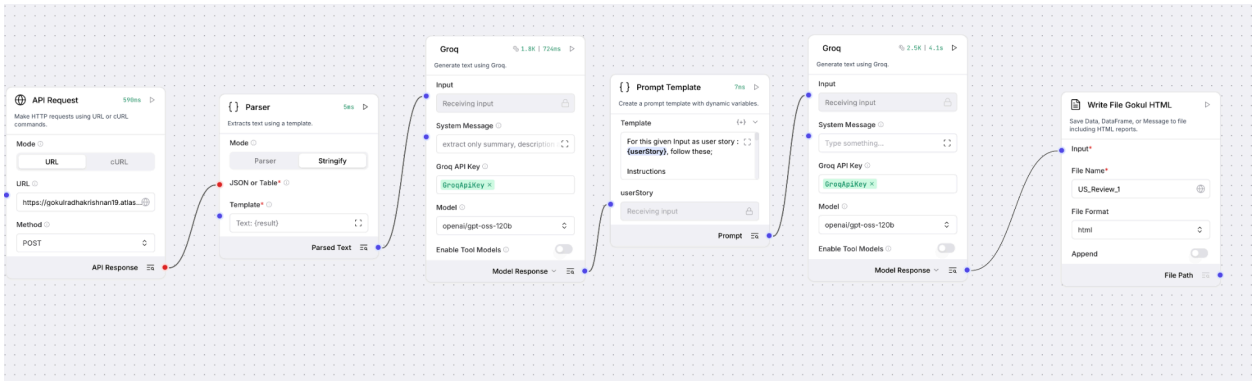
Html output created in specified path:



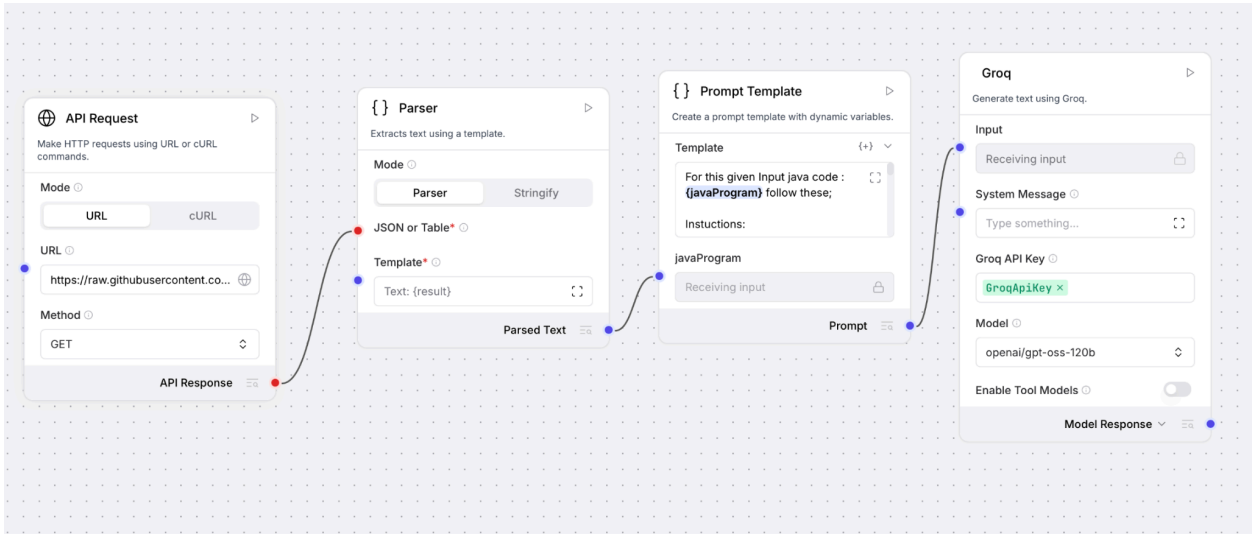
Output:



Entire Langflow:



B) Code Review Agent With Langflow:
Entire successful langflow:



Output from Groq:

Component Output

Inspect the output of the component below.

Outputs Logs

Code Review – LearnNestedFrame.java
(Selenium-based UI automation snippet)

Parameter	Description (key observations)	Score (out of 20)
Code Readability	• Naming follows Java conventions (`LearnNestedFrame`, `driver`, `element`). • Indentation is consistent and the code is short enough to follow visually. • No Javadoc or inline comments explaining *why* the frames are switched, what the test validates, or any pre-conditions. • The XPath string is escaped and a bit hard to read; a constant or a descriptive variable name would help.	**14**
Code Efficiency	• No loops or heavy processing – the snippet is already O(1). • Uses an implicit wait of 15 s, which can mask performance problems and slow down the test suite. • Switching to frames by index (`frame(0)`) is brittle; locating by a stable attribute would be more reliable.	**15**
Error Handling	• No `try-catch` / `finally` block; any `NoSuchElementException`, `TimeoutException`, or driver-initialisation failure will abort the JVM. • No logging or meaningful error messages. • The driver is never closed (`driver.quit()`), leading to orphaned browser processes.	**5**
Maintainability	• All logic lives in a single `main` method – not reusable or extensible. • Hard-coded URL, locators, and frame indexes make future changes error-prone. • No Page Object Model (POM) abstraction, no test framework (TestNG/JUnit) integration, and no configuration handling.	**8**
Best Practices & Standards	• Missing **SOLID** considerations – the class does a single thing (launch & click) but mixes driver lifecycle, navigation, and test actions. • No use of **DRY** – the same driver setup would be duplicated across tests. • No explicit `System.setProperty` for the Chrome driver binary (relies on WebDriverManager or system PATH). • No unit/integration test scaffolding, no assertions, and no reporting. • Imports are correct but could be trimmed if a test framework is introduced.	**6**

Overall Score
48 / 100

Component Output

Inspect the output of the component below.

Outputs Logs

Actionable Feedback & Recommendations

Area	Recommendation	Why it matters
Driver Lifecycle	Wrap driver usage in a `try-finally` (or try-with-resources) block and call `driver.quit()` in the `finally`.	Guarantees browser cleanup, prevents resource leaks on failures.
Exception Management	Catch Selenium-specific exceptions (`NoSuchElementException`, `TimeoutException`) and log a clear message (use SLF4J/Log4j).	Improves debuggability and allows the test suite to continue gracefully.
Wait Strategy	Replace the global implicit wait with explicit `WebDriverWait` for the specific elements/frames you interact with.	Reduces unnecessary waiting, makes failures faster and more deterministic.
Locators & Frame Switching	Store locators as `private static final By` constants or use a Page Object. Prefer locating frames by a stable attribute (`id`, `name`) instead of index.	Improves readability, eases maintenance when UI changes, and reduces flakiness.
Modularisation	Extract the following responsibilities into separate methods or classes: 1. `setupDriver()` – driver init & config 2. `navigateToPage()` – URL loading 3. `switchToNestedFrame()` – frame handling 4. `clickButton()` – action 5. `tearDown()` – quit driver.	Encourages reuse, simplifies future extensions (e.g., adding more test steps).
Test Framework Integration	Convert the `main` method into a TestNG/JUnit test method annotated with `@Test`. Use `@BeforeClass/@AfterClass` for setup/teardown.	Enables parallel execution, reporting, and CI/CD integration.
Configuration Management	Externalise URL, timeouts, and driver binary path to a properties/YAML file or environment variables.	Allows the same code to run across environments (dev, QA, prod) without code changes.
Documentation	Add Javadoc for the class and public methods, and inline comments describing the purpose of each frame switch.	Helps new team members understand intent quickly.
Logging	Introduce a logger (`private static final Logger LOG = LoggerFactory.getLogger(LearnNestedFrame.class);`) and log key steps (driver start, navigation, frame switches, click action).	Provides runtime visibility and aids troubleshooting.
Static Analysis & Formatting	Run tools like SpotBugs, Checkstyle, and Google Java Format to enforce coding standards automatically.	Guarantees consistent style across the repository.

Summary

The submitted snippet demonstrates a **working Selenium flow** for interacting with a nested frame, but it is **far from production-ready**. While the code is syntactically correct and readable at a glance, it lacks the structural, defensive, and configurational qualities expected in a shared automation repository.

Key gaps are:

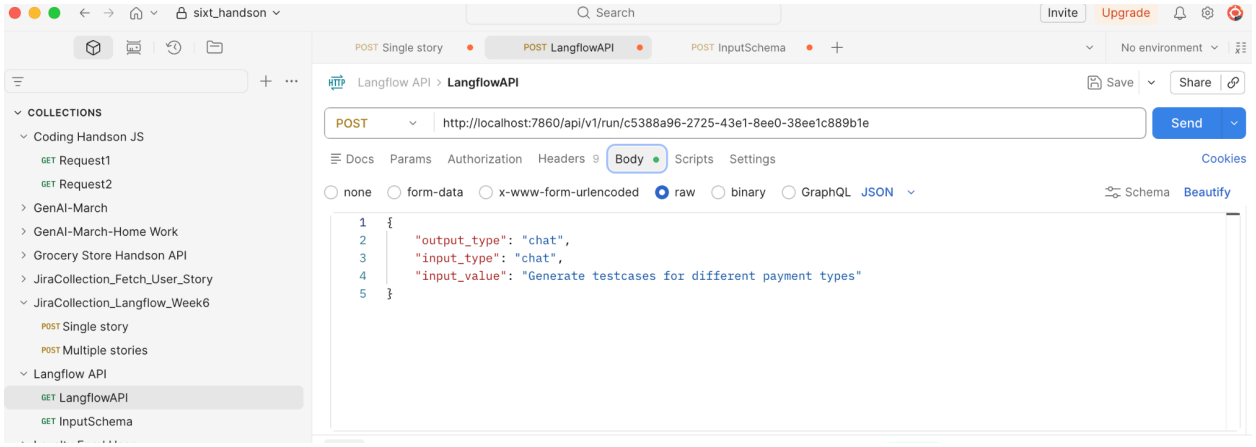
* **No error handling or resource cleanup** – risking orphaned browsers and obscure failures.
* **Hard-coded values and brittle locators** – making future UI changes painful.
* **Absence of a test framework and modular design** – limiting reusability and CI integration.
* **Missing logging and documentation** – reducing maintainability for other developers.

By refactoring the code into a **Page Object Model**, integrating it with **TestNG/JUnit**, adding **explicit waits**, **proper exception handling**, and **external configuration**, the overall quality can be lifted dramatically—potentially moving the score into the 80-90 range.

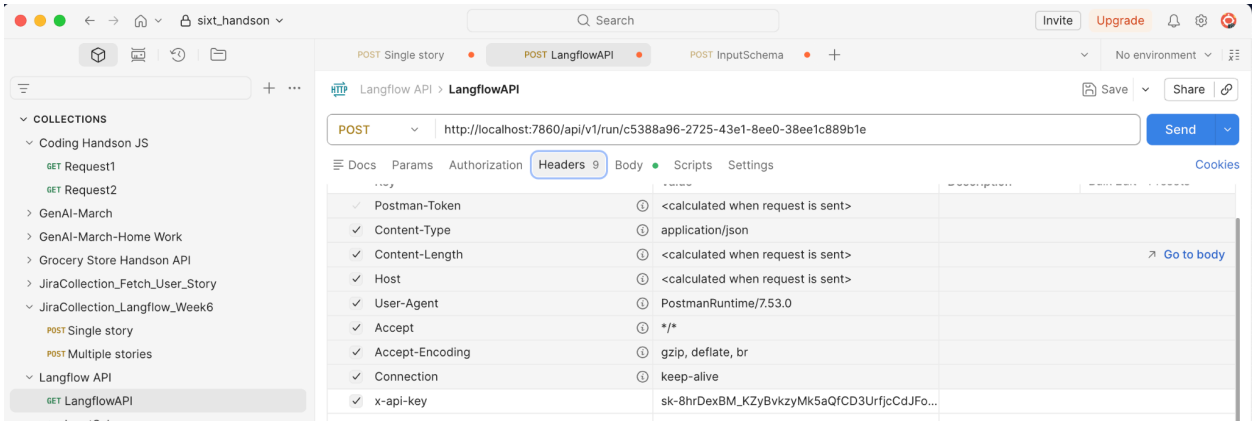
Implement the recommendations above, and the class will evolve from a simple demo script into a robust, maintainable component of the automation suite.

Close

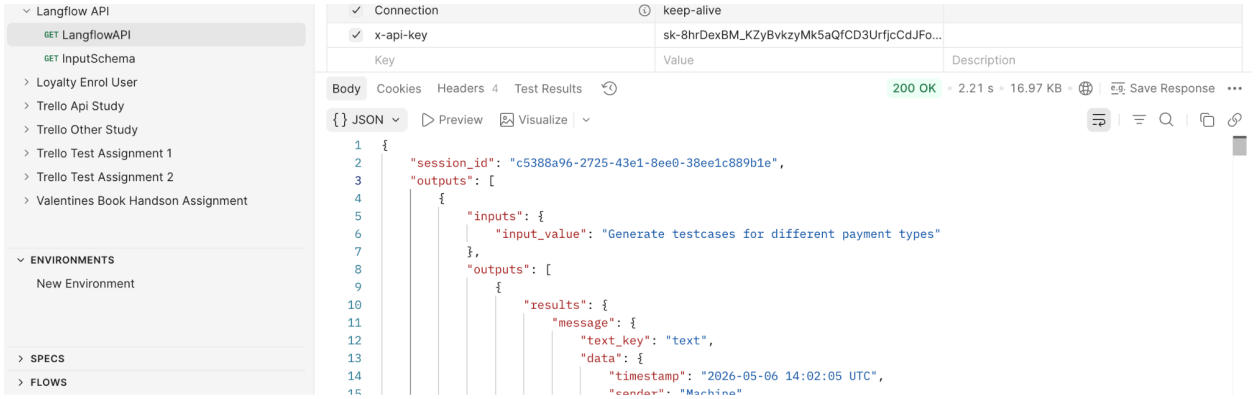
Langflow Api handson with Test Case Generator:



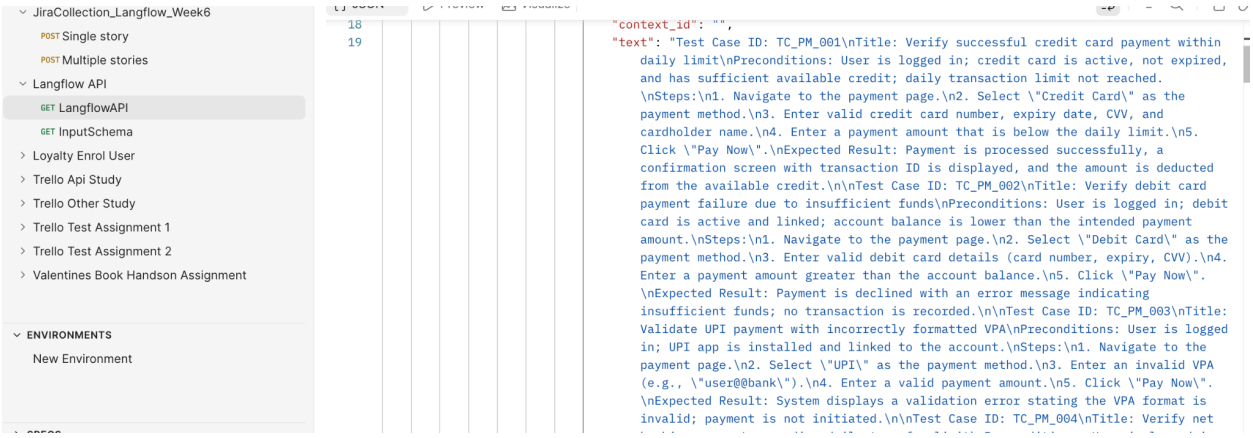
Header Details:



Output Response:



Output Response Expanded for Langflow Api:



Input Schema POST request with test case generator prompt and response received: